

JUPSOFT SYSTEMS PVT. LTD.

Technology · Cloud · AI · Digital Transformation

Centralized Multi-Site Blog Management Platform

Technical Requirement & Developer Specification

Next.js · NestJS (Node.js) · PostgreSQL · Redis · AWS S3

Version 1.0

Audience: Engineering Team · Status: Ready for Sprint Planning

Table of Contents

Table of Contents.....	2
1. Project Overview	3
2. Business Objective	3
3. System Architecture	3
4. Technology Stack	4
5. User Roles & Permissions	4
6. Website Management Module	5
7. Blog Management Module	5
8. Multi-Language Support	6
9. Categories & Taxonomy	6
10. Media Management Module	6
11. SEO Module.....	6
12. REST API Architecture	7
13. Website Integration (Hybrid Method)	8
14. Analytics Module	9
15. Security Requirements	9
16. Performance & Caching	9
17. Core Database Tables	10
18. AWS Infrastructure	10
19. Future SaaS Readiness	10
20. Development Phases	10
21. Expected Deliverables	11

1. Project Overview

This document specifies a centralized Blog Management Platform that allows content teams to create, review, schedule and publish blogs from a single administrative panel. Each blog belongs to exactly one website, and any external website can consume published content through secure REST APIs.

The platform acts as a single content hub serving multiple domains. It is designed as a product from day one — multi-tenant, scalable and SaaS-ready — not as a single-site CMS.

Confirmed Decisions

Decision Area	Selected Approach
Blog–Website relationship	One blog belongs to exactly one website
Authentication	JWT (short-lived access token + refresh token)
Consumer rendering (SEO)	SSR / ISR — consuming sites fetch the API at request or build time
Backend framework	NestJS (structured, modular, SaaS-ready)
Media storage	AWS S3 + CloudFront CDN
Integration method	Hybrid — REST API + Redis cache + Webhook revalidation
Public URL structure	/blog/{slug} on each consuming site
Hosting	AWS (ECS, RDS PostgreSQL, ElastiCache, S3, CloudFront)

2. Business Objective

Current Challenge

Blogs are currently managed separately per website, which produces:

- Content duplication across sites
- Inconsistent SEO handling
- Multiple admin panels to maintain
- Weak content governance and no single source of truth
- No centralized analytics

Target Outcome

A single platform where writers create, editors review, publishers approve, and consuming websites fetch blogs dynamically over the API — while each website keeps its own independent content library.

Example Websites

Website	Domain	Content Library
Jupsoft	jupsoft.com	Independent
DigifyNext	digifynext.com	Independent
School ERP	schoolerp.in	Independent
Future Clients	Any domain	Independent / white-label

3. System Architecture

The platform separates the authoring plane (admin CMS + backend) from the delivery plane (public read APIs consumed by external sites). Writes go through the workflow engine; reads are served from cache and revalidated by webhooks.

High-Level Flow



Delivery Model (Hybrid)

- Consuming sites render pages with SSR/ISR and call the public API server-side — no client-side content fetch, so crawlers always receive fully rendered HTML.
- Read responses are cached in Redis at the platform and via ISR at the consuming site.
- On publish/update, a webhook triggers on-demand revalidation of the affected page on each consuming site, so cached pages refresh without waiting for TTL.

4. Technology Stack

Layer	Technology	Notes
Admin Frontend	Next.js 15 (App Router, TypeScript)	SSR, SEO-friendly, fast
UI	Tailwind CSS + ShadCN UI	Consistent enterprise UI kit
Editor	Tiptap	Rich text, structured JSON content
State / Forms	Zustand + React Hook Form	Lightweight state, validated forms
Backend	NestJS (Node.js, TypeScript)	Modular, DI, guards, SaaS-ready
Database	PostgreSQL	JSONB, full-text search, multi-language
Cache	Redis (AWS ElastiCache)	API cache, sessions, rate limits
Storage	AWS S3 + CloudFront	Media + CDN delivery
Search	PostgreSQL full-text (Phase 1)	Elastic/OpenSearch optional later
Auth	JWT access + refresh	RBAC via NestJS guards
Docs	Swagger / OpenAPI	Auto-generated API docs
Deploy	Docker + AWS ECS	Container-based, autoscaling

5. User Roles & Permissions

Access is enforced through Role-Based Access Control (RBAC). Roles are scoped per website where relevant — a user may hold different roles on different websites.

Role	Create	Edit	Review/Approve	Publish/Schedule	SEO	Manage Sites & Users
Super Admin	Yes	Yes	Yes	Yes	Yes	Yes

Role	Create	Edit	Review/Approve	Publish/Schedule	SEO	Manage Sites & Users
Content Writer	Yes	Own only	No	No	Basic	No
Editor	Yes	Assigned	Yes	No	Suggest	No
Publisher	No	No	No	Yes	No	No
SEO Manager	No	SEO fields	No	No	Yes	No

Permissions are stored as granular scopes (e.g. *blog.create*, *blog.publish*, *seo.edit*, *site.manage*) and mapped to roles, so new roles can be composed without code changes.

6. Website Management Module

Websites are first-class tenants. Each has its own API key/scope, S3 prefix, and content library.

Table: websites

```

id          UUID  PK
name        VARCHAR
domain      VARCHAR  UNIQUE
logo_url    VARCHAR
description  TEXT
api_key     VARCHAR (for consumer auth / rate scoping)
status      ENUM(active, inactive)
created_at  TIMESTAMPTZ
updated_at  TIMESTAMPTZ

```

Every blog carries a *website_id* foreign key; all list/detail APIs are filtered by website so libraries never leak across tenants.

7. Blog Management Module

Status Workflow

Draft -> Under Review -> Approved -> Scheduled -> Published -> Archived

Transitions are guarded by role: writers move Draft → Under Review; editors move Under Review → Approved (or back to Draft with change requests); publishers move Approved → Scheduled/Published. Every transition is written to *workflow_logs*.

Blog Fields

Group	Fields
Core	title, slug, excerpt, content (Tiptap JSON), featured_image, website_id, author_id
Publishing	status, publish_date, published_by, scheduled_at
SEO	meta_title, meta_description, meta_keywords, canonical_url, focus_keyword, robots
Social (OG/Twitter)	og_title, og_description, og_image, twitter_title, twitter_description, twitter_image
System	created_at, updated_at, view_count

Slug & URL Rules

- Slug is unique per website (not globally), auto-generated from the title and editable.
- Public URL on each consuming site is `/blog/{slug}`.
- Changing a published slug creates a 301 redirect record so existing links and SEO equity are preserved.

8. Multi-Language Support

Content is separated from translations. The `blogs` table holds language-independent data; `blog_translations` holds per-language content and SEO.

```
blogs
  id, website_id, status, publish_date, author_id, ...

blog_translations
  id, blog_id (FK), language_code,
  title, slug, content, excerpt,
  meta_title, meta_description
  UNIQUE(blog_id, language_code)
  UNIQUE(website_id, language_code, slug)
```

Initial languages: en, hi, fr, ar. API accepts a `lang` parameter and falls back to the website's default language when a translation is missing.

9. Categories & Taxonomy

Categories and tags are scoped per website and linked to blogs through many-to-many join tables.

Table	Purpose
<code>categories</code>	<code>id, website_id, name, slug, parent_id (nested), description</code>
<code>tags</code>	<code>id, website_id, name, slug</code>
<code>blog_categories</code>	<code>blog_id, category_id (M:N)</code>
<code>blog_tags</code>	<code>blog_id, tag_id (M:N)</code>

Example categories: Technology, AI, Digital Transformation, ERP, Cloud, Marketing. Example tags: AI, School ERP, Automation, CRM, Cloud.

10. Media Management Module

All media is uploaded to AWS S3 and served through CloudFront. Uploads are processed for web delivery.

Pipeline

- Upload to S3 (server-issued pre-signed URL)
- Compress and convert to WebP
- Generate thumbnail + responsive sizes
- Store metadata in `media_library`
- Return CDN URL
- Soft-delete with lifecycle cleanup

S3 Layout

```
s3://<bucket>/blogs/<website>/<yyyy>/<mm>/<file>
e.g. s3://jupsoft-blogs/blogs/jupsoft/2026/09/erp-hero.webp
```

Table: `media_library`

```
id, website_id, file_name, file_type, file_size, s3_key, cdn_url, alt_text, uploaded_by,
created_at
```

11. SEO Module

SEO is a mandatory, validated part of every blog. The delivery model (SSR/ISR on consuming sites) guarantees crawlers receive complete metadata and rendered content.

Mandatory Fields

- Meta title, meta description, meta keywords
- Canonical URL, focus keyword, slug, robots directive
- Open Graph + Twitter card fields for social sharing

Automated SEO Checks (score 0–100)

Check	Rule
Meta title length	50–60 characters
Meta description length	140–160 characters
Focus keyword	Present in title, slug, first paragraph
Image alt tags	All images have alt text
Heading structure	Exactly one H1; logical H2/H3
Canonical	Set and valid

The API also exposes canonical and hreflang data so consuming sites can emit correct <head> tags and multi-language alternates.

12. REST API Architecture

Public (read) endpoints serve published content and are cached. Admin (write) endpoints require JWT + RBAC. Base URL:

```
https://api.<platform-domain>.com/v1/
```

Public Endpoints

Method / Path	Purpose
GET /blogs?website=&category=&tag=&lang=&page=	Paginated list, filtered by website
GET /blogs/{slug}?website=&lang=	Single blog by slug (per website)
GET /blogs/latest?website=	Latest published blogs
GET /blogs/popular?website=	Most-viewed blogs
GET /categories?website=	Category tree
GET /tags?website=	Tags
GET /search?website=&q=	Full-text search

Example: Single Blog Response

```
GET /v1/blogs/erp-for-schools?website=jupsoft&lang=en

{
  "id": "...",
  "website": "jupsoft",
  "slug": "erp-for-schools",
  "title": "ERP for Schools",
  "excerpt": "...",
  "content": { /* Tiptap JSON or rendered HTML */ },
  "featured_image": "https://cdn....webp",
```

```

"seo": {
  "meta_title": "...",
  "meta_description": "...",
  "canonical_url": "https://jupsoft.com/blog/erp-for-schools",
  "focus_keyword": "school erp",
  "robots": "index,follow"
},
"og": { "og_title": "...", "og_image": "..." },
"categories": ["ERP"],
"tags": ["School ERP", "Automation"],
"publish_date": "2026-09-01T10:00:00Z"
}

```

Admin Endpoints (JWT + RBAC)

Method / Path	Purpose
POST /admin/blogs	Create draft
PUT /admin/blogs/{id}	Update blog
POST /admin/blogs/{id}/submit	Submit for review
POST /admin/blogs/{id}/approve	Approve (editor)
POST /admin/blogs/{id}/publish	Publish / schedule (publisher)
POST /admin/media	Request pre-signed upload
POST /admin/websites	Manage websites (super admin)

13. Website Integration (Hybrid Method)

Consuming sites use SSR/ISR + Redis-cached API + webhook revalidation. This is the confirmed integration model and gives the best SEO with fast, always-fresh pages.

Step 1 — Server-side fetch (Next.js on consumer site)

```

// app/blog/[slug]/page.tsx (consumer website)
export const revalidate = 3600; // ISR fallback TTL

export async function generateMetadata({ params }) {
  const blog = await fetch(
    `https://api.platform.com/v1/blogs/${params.slug}?website=jupsoft`,
    { next: { tags: ['blog:' + params.slug] } }
  ).then(r => r.json());
  return {
    title: blog.seo.meta_title,
    description: blog.seo.meta_description,
    alternates: { canonical: blog.seo.canonical_url },
    openGraph: { title: blog.og.og_title, images: [blog.og.og_image] },
  };
}

export default async function BlogPage({ params }) {
  const blog = await fetch(
    `https://api.platform.com/v1/blogs/${params.slug}?website=jupsoft`,
    { next: { tags: ['blog:' + params.slug] } }
  ).then(r => r.json());
  return <Article blog={blog} />;
}

```

Step 2 — Platform caches reads in Redis

The public API checks Redis first (key: blog:{website}:{slug}:{lang}); on miss it reads PostgreSQL and populates the cache. This absorbs traffic spikes and keeps response times under ~300ms.

Step 3 — Webhook revalidation on publish

```
POST https://jupsoft.com/api/revalidate      (per-site endpoint)
Header: X-Signature: <HMAC of body>
{
  "event": "blog.published",      // or blog.updated / blog.unpublished
  "website": "jupsoft",
  "slug": "erp-for-schools"
}

// consumer handler
revalidateTag('blog:' + body.slug);
```

- On publish/update/unpublish the platform invalidates its Redis key and fires the webhook.
- The consumer verifies the HMAC signature, then revalidates just the affected page/tag.
- If the webhook fails, the ISR TTL still refreshes the page as a fallback — no page ever goes stale indefinitely.

14. Analytics Module

Scope	Metrics
Blog	Views, unique visitors, avg. read time, bounce rate
Website	Total blogs, published blogs, top categories, traffic trend
Author	Blogs written, published count, total views
Dashboard	Top blogs, top authors, traffic over time

View events are recorded via a lightweight tracking endpoint (or edge log ingestion) and aggregated asynchronously so they never slow page delivery.

15. Security Requirements

Area	Requirement
Authentication	JWT access (short TTL) + refresh token rotation
Passwords	Bcrypt hashing, strength policy
Authorization	RBAC via NestJS guards, per-website scoping
API protection	Rate limiting, per-website API keys, CORS allowlist
Webhooks	HMAC-signed payloads, replay protection
Auditing	Login history, workflow + audit logs, IP logging
Web	CSRF protection, XSS sanitization of editor content
Transport	HTTPS/TLS everywhere, secrets in AWS Secrets Manager

16. Performance & Caching

- Redis caches blog detail, category lists and homepage blog lists; invalidated on publish/update.
- CloudFront CDN fronts media and (optionally) API GET responses.
- Images served as WebP with responsive sizes and lazy loading.
- Consumer pages use ISR so most requests are served as static HTML.

Target API response time: **< 300ms**

17. Core Database Tables

users	roles	permissions
websites		
blogs	blog_translations	
categories	tags	
blog_categories	blog_tags	
media_library		
workflow_logs	seo_audit_logs	
analytics	api_logs	
webhooks	redirects	

18. AWS Infrastructure

```

Route 53
|
CloudFront CDN -----> S3 (media)
|
Application Load Balancer
|
ECS (NestJS backend, autoscaling)
|
+--> RDS PostgreSQL (Multi-AZ)
+--> ElastiCache Redis
+--> S3 (media origin)
+--> SES (transactional email)
+--> CloudWatch (logs, metrics, alarms)

```

Service	Role
AWS ECS	Containerized NestJS backend (preferred)
AWS RDS (PostgreSQL)	Primary datastore, multi-AZ
AWS ElastiCache (Redis)	Cache + sessions + rate limiting
AWS S3	Media storage / CDN origin
AWS CloudFront	CDN for media and API reads
AWS SES	Notifications & workflow emails
AWS CloudWatch	Monitoring, logging, alarms

19. Future SaaS Readiness

The architecture is multi-tenant from the start. The following extensions require no structural rework:

- Multiple organizations / white-label deployments
- Subscription plans and API-usage billing
- Custom domains per tenant
- AI content generation and AI SEO suggestions
- AI translation for blog_translations
- Content syndication across tenants

20. Development Phases

Phase	Scope	Timeline
1 — Core CMS	Auth (JWT), website management, blog CRUD, categories, tags, media library	3–4 weeks
2 — Workflow & SEO	Approval workflow, scheduling, SEO module + score	2 weeks
3 — Public APIs	Read APIs, API auth/keys, Swagger docs	1–2 weeks
4 — Analytics	Tracking, dashboards, reports	2 weeks
5 — Scale & Integration	Redis, CloudFront, webhooks, revalidation, tuning	1 week

21. Expected Deliverables

1. Next.js admin portal (App Router, TypeScript)
2. NestJS REST API backend (modular, guarded)
3. PostgreSQL schema + migrations
4. AWS S3 media integration with WebP pipeline
5. SEO management module with scoring
6. Multi-language (blog_translations) module
7. Workflow engine with role-based transitions
8. Analytics dashboard
9. Swagger / OpenAPI documentation
10. Webhook + revalidation framework
11. Docker setup + AWS deployment scripts
12. RBAC implementation
13. Production-ready source code + technical documentation

Prepared for the Jupsoft Systems engineering team · Version 1.0 · Ready for architecture design, database modeling, sprint planning and development.